Beginning Tapestry: Java Web Components

Howard M. Lewis Ship

hlship@apache.org

View from 20,000 Feet

- JSPs are a Nightmare!
  - One file …many languages!
    - HTML
    - Java Scriptlets <% … %>
    - Client-side JavaScript
    - JSP directives
    - JSP tags
  - Enough to make you blind!

View from 20,000 Feet

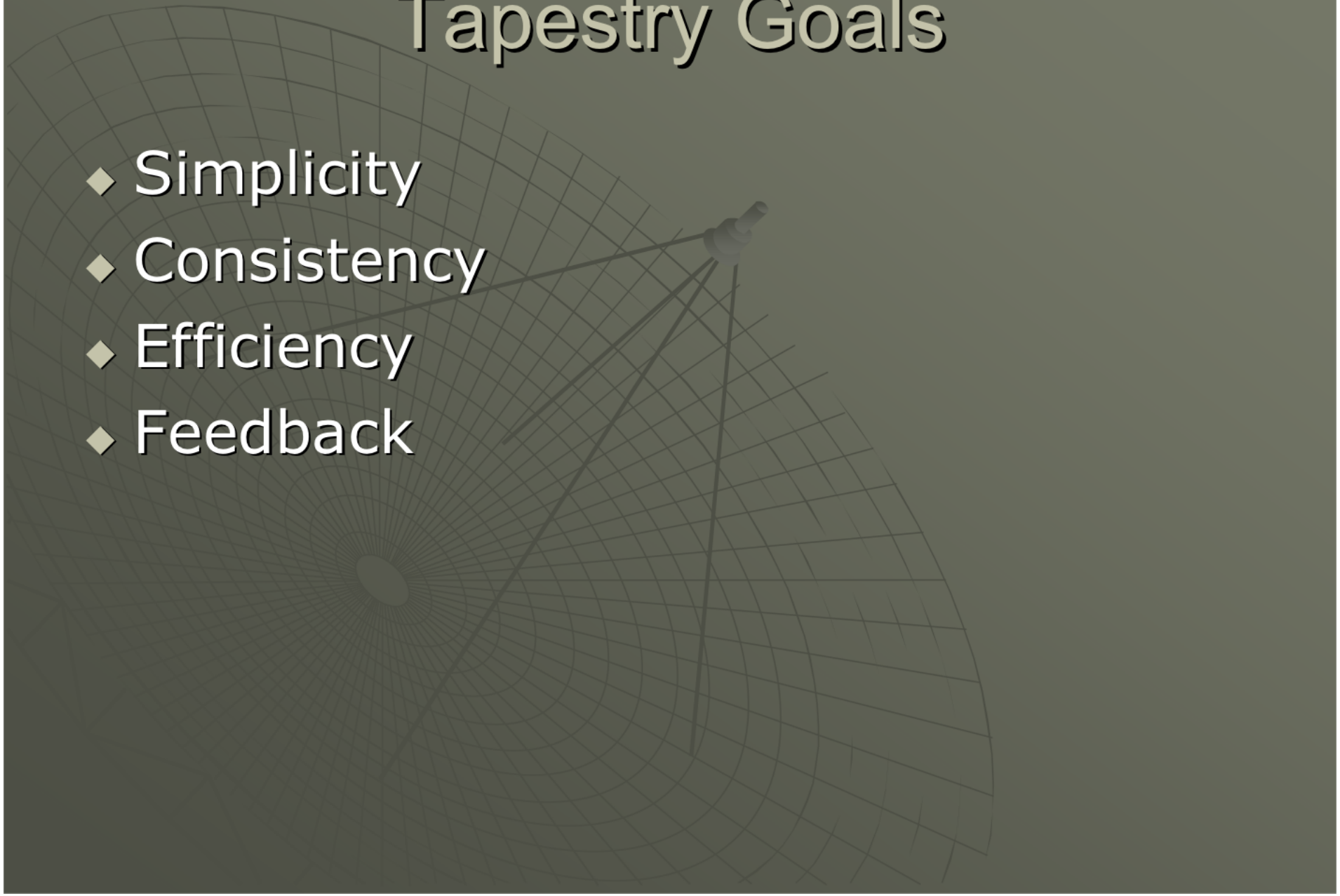
- Servlets and JSPs weakly bound
- Hard to apply team development … no standards
- Have to write lots of mechanical, repetitive code
- Doesn't feel like O-O ...
  - Only one operation per class?
  - Logic here (servlet), state there (HttpSession)?
  - Inheritance? Aggregation? Reuse?

Tapestry Overview

- Provides *structure* to replace kludges and tricks
- Very much O-O: inheritance, aggregation, interfaces, encapsulation
- Everything's a component
- Best practices rolled into framework
- Very team friendly
- Less Code == Less Bugs

Tapestry Goals

- Simplicity
- Consistency
- Efficiency
- Feedback



Objects, Methods and Properties

- "Mantra" of Tapestry
- Don't treat web apps like a text processing problem!
- JavaBeans properties
  - Sources of properties (domain objects)
  - Readers/writers of properties (components)

Model-View-Controller

- Domain Objects (Model)
  - Look like bundle of properties to read and update
- HTML template (View)
  - No Java code ever
  - Looks like HTML, tastes like HTML
  - Still previews correctly
  - Minimal additions / almost invisible

Model-View-Controller

- Pages / Components (Controller)
  - Specification (in XML)
    - Links together placeholders (in the template) with rest of application, components, logic
    - May be very simple (or skipped for trivial pages)
    - Spindle Eclipse plugin if you don’t like XML
  - Java Class
    - Stores transient & persistent properties
    - Contains business logic

Tapestry in Action

A quick example … with a few bells and whistles

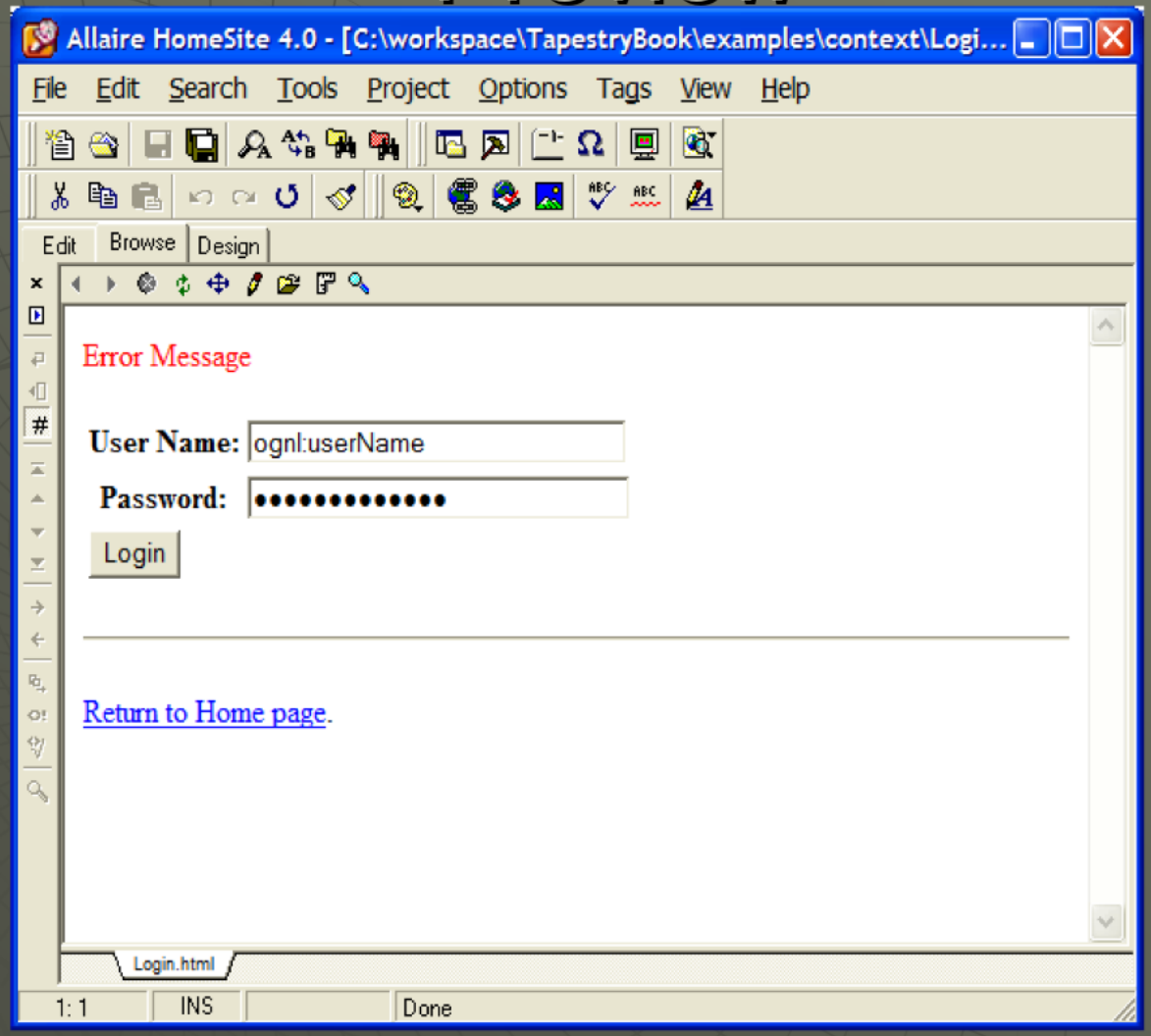
Tapestry in Action

- Standard "Login" page
  - User name
  - Password (hidden)
  - Error message (on failure)
- Use just basic components

HTML Template

```
<html>
<body>
Login:
<span jwcid="@Conditional" condition="ognl:message">
<p><font color="red"><span jwcid="@Insert" value="ognl:message">Error Message</span></font>
</span>
<p>
<form jwcid="@Form" listener="ognl:listeners.login">
<table border="0">
<tr><th>User:</th>
    <td><input type="text" jwcid="@TextField" value="ognl:userName" size="30"/></td>
</tr>
<tr><th>Password:</th>
    <td><input type="password" jwcid="@TextField"
          value="ognl:password" hidden="ognl:true" size="30"/></td>
</tr>
<tr><td><input type="submit" value="Login"></td></tr>
</table>
</form>
</body>
</html>
```

HTML Template – WYSIWYG Preview



HTML Template

```
<html>
<body>
Login:
<span jwcid="@Conditional" condition="ognl:message">
<p><font color="red"><span jwcid="@Insert" value="ognl:message">Error Message</span></font>
</span>
<p>
<form jwcid="@Form" listener="ognl:listeners.login">
<table border="0">
<tr><th>User:</th>
    <td><input type="text" jwcid="@TextField" value="ognl:userName" size="30"/></td>
</tr>
<tr><th>Password:</th>
    <td><input type="password" jwcid="@TextField"
          value="ognl:password" hidden="ognl:true" size="30"/></td>
</tr>
<tr><td><input type="submit" value="Login"></td></tr>
</table>
</form>
</body>
</html>
```

jwcid – Java Web Component Id

Identifies *component* within template

Components may enclose HTML and other components

HTML Template

```
<html>
<body>
Login:
<span jwcid="@Conditional" condition="ognl:message">
<p><font color="red"><span jwcid="@Insert" value="ognl:message">Error Message</span></font>
</span>

<p>
<form jwcid="@Form" listener="ognl:listeners.login">
<table border="0">
<tr><th>User:</th>
    <td><input type="text" jwcid="@TextField" value="ognl:userName" size="30"/></td>
</tr>
<tr><th>Password:</th>
    <td><input type="password" jwcid="@TextField"
          value="ognl:password" hidden="ognl:true" size="30"/></td>
</tr>
<tr><td><input type="submit" value="Login"></td></tr>
</table>
</form>
</body>
</html>
```

@*Type* – Defines the type of component

These are anonymous (Tapestry provides a unique id)

Conditional, Insert, Form, TextField – built in components

HTML Template

```
<html>
<body>
Login:
<span jwcid="@Conditional" condition="ognl:message">
<p><font color="red"><span jwcid="@Insert" value="ognl:message">Error Message</span></font>
</span>
<p>
<form jwcid="@Form" listener="ognl:listeners.login">
<table border="0">
<tr><th>User:</th>
    <td><input type="text" jwcid="@TextField" value="ognl:userName" size="30"/></td>
</tr>
<tr><th>Password:</t
    <td><input type=
        value="ogn
</tr>
<tr><td><input type=
</table>
</form>
</body>
</html>
```

condition is a *parameter* of Conditional

Body of Conditional rendered only if true

message is an OGNL expression

- **OGNL – Object Graph Navigation Language**
- **Strings are true if not null or empty**

"If there's a message to display, then ..."

HTML Template

```
<html>
<body>
Login:
<span jwcid="@Conditional" condition="ognl:message">
<p><font color="red"><span jwcid="@Insert" value="ognl:message">Error Message</span></font>
</span>
<p>
<form jwcid="@Form" listener="ognl:listeners.login">
<table border="0">
<tr><th>User:</th>
    <td><input type="text" jwcid=                                    td>
</tr>
<tr><th>Password:</th>
    <td><input type="password" jw
          value="ognl:password" h
</tr>
<tr><td><input type="submit" value= Login ></td></tr>
</table>
</form>
</body>
</html>
```

Insert _ inserts text into response

value parameter is text to insert

Automatically converts '<' to '<', etc.

HTML Template

```
<html>
<body>
Login:
<span jwcid="@Conditional" condition="ognl:message">
<p><font color="red"><span jwcid="@Insert" value="ognl:message">Error Message</span></font>
</span>
<p>
<form jwcid="@Form" listener="ognl:listeners.login">
<table border="0">
<tr><th>User:</th>
    <td><input type="text" jwcid=
</tr>
<tr><th>Password:</th>
    <td><input type="password" jw
        value="ognl:password" h
</tr>
<tr><td><input type="submit" valu
</table>
</form>
</body>
</html>
```

Body of Insert always *discarded*

Text is allowed for *previewing*

HTML producers can see what page looks like without running the application

HTML Template

```
<html>
<body>
Login:
<span jwcid="@Conditional" condition="ognl:message">
<p><font color="red"><span jwcid="@Insert" value="ognl:message">Error Message</span></font>
</span>

<p>
<form jwcid="@Form" listener="ognl:listeners.login">
<table border="0">
<tr><th>User:</th>
    <td><input type="text" jwcid="@TextField" value="ognl:userName" size="30"/></td>
</tr>
<tr><th>Password:</th>
    <td><input type="password" jwcid="@TextField"
          value="ognl:passw
</tr>
<tr><td><input type="submit
</table>
</form>
</body>
</html>
```

Renders HTML <form>

Form element components must be enclosed by Form

TextField, Checkbox, Radio, etc.

listener is invoked when form is submitted

Application logic provided in Java class

Look Ma! No URLs!

HTML Template

```
<html>
<body>
Login:
<span jwcid="@Conditional" condition="ognl:message">
<p><font color="red"><span jwcid="@Insert" value="ognl:message">Error Message</span></font>
</span>

<p>
<form jwcid="@Form" listener="ognl:listeners.login">
<table border="0">
<tr><th>User:</th>
    <td><input type="text" jwcid="@TextField" value="ognl:userName" size="30"/></td>
</tr>
<tr><th>Password:</th>
    <td><input type="password" jwcid="@TextField"
          value="ognl:passw
</tr>
<tr><td><input type="submit
</table>
</form>
</body>
</html>
```

listeners.login is shorthand for

public void login(IRequestCycle cycle)

Little bit of reflection magic

Property "login" of property "listeners" of page

Listener invoked *last* after all properties updated

HTML

TextField reads/updates a string property

userName property of page is:

- read on render
- updated on submit

Look Ma! No Query Parameters!

```
<html>
<body>
Login:
<span jwcid="@Conditional" condition="ogn
<p><font color="red"><span jwcid="@Insert
</span>
<p>
<form jwcid="@Form" listener="ognl:listeners.login">
<table border="0">
<tr><th>User:</th>
    <td><input type="text" jwcid="@TextField" value="ognl:userName" size="30"/></td>
</tr>
<tr><th>Password:</th>
    <td><input type="password" jwcid="@TextField"
          value="ognl:password" hidden="ognl:true" size="30"/></td>
</tr>
<tr><td><input type="submit" value="Login"></td></tr>
</table>
</form>
</body>
</html>
```

mplate

- "30" is a static string
  - not an expression, no "ognl:" prefix
- size *is not* a parameter of TextField
- Still shows up in response page – as an *informal parameter*
- Useful for things like CSS class
- Informal parameters *can be* expressions

```
gnl:message">Error Message</span></font>
<form jwcid="@Form" listener="ognl:listeners.login">
<table border="0">
<tr><th>User:</th>
    <td><input type="text" jwcid="@TextField" value="ognl:userName" size="30"/></td>
</tr>
<tr><th>Password:</th>
    <td><input type="password" jwcid="@TextField"
          value="ognl:password" hidden="ognl:true" size="30"/></td>
</tr>
<tr><td><input type="submit" value="Login"></td></tr>
</table>
</form>
</body>
</html>
```

HTML Template

```
<html>
<body>
Login:
<span jwcid="@Conditional" condition="ognl:m
<p><font color="red"><span jwcid="@Insert" v
</span>
<p>
<form jwcid="@Form" listener="ognl:listeners
<table border="1">
<tr><th>User:</th>
    <td><input type="text" jwcid="@TextField" value="ognl:userName" size="30"/></td>
</tr>
<tr><th>Password:</th>
    <td><input type="password" jwcid="@TextField"
          value="ognl:password" hidden="ognl:true" size="30"/></td>
</tr>
<tr><td><input type="submit" value="Login"></td></tr>
</table>
</form>
</body>
</html>
```

- Component tags must balance
- Use XML-style for empty
- Rest of template is freeform
- Tapestry *doesn't care* what tags are!
  - Could be or <fred>
  - This way good for previewing

HTML Template

```
<html>
<body>
Login:
<span jwcid="@Conditi
<p><font color="red">                                    r Message</span></font>
</span>

<p>
<form jwcid="@Form" l
<table border="0">
<tr><th>User:</th>
    <td><input type="text" jwcid="@TextField" value="ognl:userName" size="30"/></td>
</tr>
<tr><th>Password:</th>
    <td><input type="password" jwcid="@TextField"
          value="ognl:password" hidden="ognl:true" size="30"/></td>
</tr>
<tr><td><input type="submit" value="Login"></td></tr>
</table>
</form>
</body>
</html>
```

TextField for both normal and password

hidden parameter to select between the two

HTML Template

```
<html>
<body>
Login:
<span jwcid="@Conditional" condition="ognl:message">
<p><font color="red"><span jwcid="@Insert" value="ognl:message">Error Message</span></font>
</span>

<p>
<form jwcid="@Form" listener="ognl:listeners.login">
<table border="0">
<tr><th>User:</th>
    <td><input type="text" jwcid="@TextField" value="ognl:userName" size="30"/></td>
</tr>
<tr><th>Password:</th>
    <td><input type="password" jwcid="
        value="ognl:password
</tr>
<tr><td><input type="submit" value="Lo
</table>
</form>
</body>
</html>
```

type attribute is "reserved"

- **Value in template is ignored, discarded**
- **Wont become an informal parameter**
- **There just to support WYSIWYG**

Component spec lists parameters, reserved attributes

Page Specification

```
<?xml version="1.0"?>
<!DOCTYPE page-specification PUBLIC
   "-//Apache Software Foundation//Tapestry Specification 3.0//EN"
   "http://jakarta.apache.org/tapestry/dtd/Tapestry_3.0.dtd">

<page-specification class="example.Home">

  <property-specification name="message" type="java.lang.String"/>
  <property-specification name="userName" type="java.lang.String"/>
  <property-specification name="password" type="java.lang.String"/>

</page-specification>
```

Page Specification

```
<?xml version="1.0"?>
<!DOCTYPE page-specification PUBLIC
   "-//Apache Software Foundation//Tapestry Specification 3.0//EN"
   "http://jakarta.apache.org/tapestry/dtd/Tapestry_3.0.dtd">

<page-specification class="example.Home">

  <property-specification name="message" type="java.lang.String"/>
  <property-specification
  <property-specification

</page-specification>
```

Java class Tapestry will instantiate for page

Standard naming:

- **Specification: WEB-INF/*Foo*.page**
- **HTML Template: *Foo*.html**
- **Class: *package.Foo***

Page Specification

Creates a JavaBean property

Tapestry creates subclass with fields, methods

Needed to *create* property, not *reference* existing property

Optional: can do it all in your own code, but why bother?

```
<?xml ver
<!DOCTY
   "-//Apa
   "http://j

<page-specification class="example.Home">

  <property-specification name="message" type="java.lang.String"/>
  <property-specification name="userName" type="java.lang.String"/>
  <property-specification name="password" type="java.lang.String"/>

</page-specification>
```

Java Class

```
package example;

import org.apache.tapestry.IRequestCycle;
import org.apache.tapestry.html.BasePage;

public abstract class Home extends BasePage
{
  public abstract void setMessage(String message);

  public abstract String getUserName();

  public abstract String getPassword();

  public void login(IRequestCycle cycle)
  {
    if (validLogin( getUserName(), getPassword()))
      cycle.activate("Main");
    else
      setMessage("Invalid user name or password.");
  }

  private boolean validLogin(String userName, String password)
  {
        // Talk to some backend system ...
  }
}
```

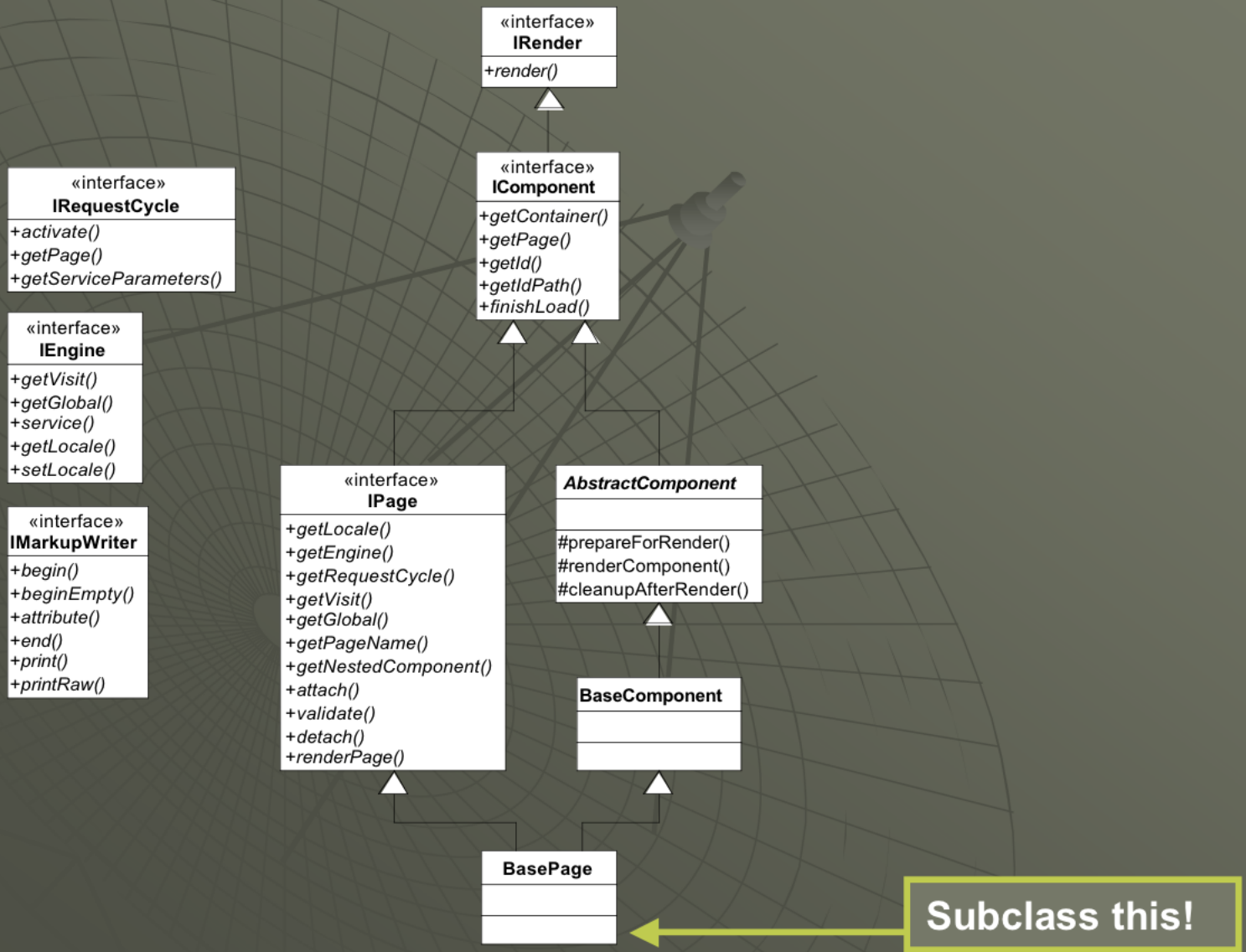
Java Class

```
package example;

import org.apache.tapestry.IRequestCycle;
import org.apache.tapestry.html.BasePage;

public abstract class Home extends BasePage
{
  public abstract void setMessage(String message);

  public abstract String

  public abstract String

  public void login(IRequestCycle cycle)
  {
    if (validLogin( getUserName(), getPassword()))
      cycle.activate("Main");
    else
      setMessage("Invalid user name or password.");
  }

  private boolean validLogin(String userName, String password)
  {
        // Talk to some backend system ...
  }
}
```

BasePage provided by framework

Used for HTML pages

Basic Classes and Interfaces



Java Class

```
package example;

import org.apache.tapestry.IRequestCycle;
import org.apache.tapestry.html.BasePage;

public abstract class Home extends BasePage
{
  public abstract void setMessage(String message);

  public abstract String getUserName();

  public abstract String getPassword();

  public void login(IRequestCycle cycle)
  {
    if (validLogin( getUserName(), getPassword()))
      cyc
    else
      set
  }

  private
  {
      /
  }
}
```

- Abstract class, abstract accessors
- Remember the page specification?
- Just the ones needed in our code
- Tapestry implements read/write accessors in subclass

Java Class

Listener method

ognl:listeners.login

Invoked when form submitted

Invoked last, after properties set

```
cle;
age;

sePage

ing message);

    public abstract String getUserName();

    public abstract String getPassword();

    public void login(IRequestCycle cycle)
    {
      if (validLogin( getUserName(), getPassword()))
        cycle.activate("Main");
      else
        setMessage("Invalid user name or password.");
    }

    private boolean validLogin(String userName, String password)
    {
          // Talk to some backend system ...
    }
}
```

Java Class

```
package example;

import org.apache.tapestry.IRequestCycle;
import org.apache.tapestry.html.BasePage;

public abstract class Home extends BasePage
{
  public abstract void setMessage(String message);

  public abstract String getUserName();

  public abstract String getPassword();

  public void login(IRequestCycle cycle)
  {
    if (validLogin( getUserName(), getPassword()))
      cycle.activate("Main");
    else
      setMessage("Invalid user name or password.");
  }

  private boolean validLogin(String userName, String password)
  {
        // Talk to some backend system ...
  }
}
```

This is where we use those accessors

Java Class

```
package example;

import org.apache.tapestry.IRequestCycle;
import org.apache.tapestry.html.BasePage;

public abstract class Home extends BasePage
{
  public abstract String getPassword();

  public void login(IRequestCycle cycle)
  {
    if (validLogin( getUserName(), getPassword()))
      cycle.activate("Main");
    else
      setMessage("Invalid user name or password.");
  }

  private boolean validLogin(String userName, String password)
  {
        // Talk to some backend system ...
  }
}
```

On login success, *activate* a new page to render the response

Real app would do additional bookkeeping first

Java Class

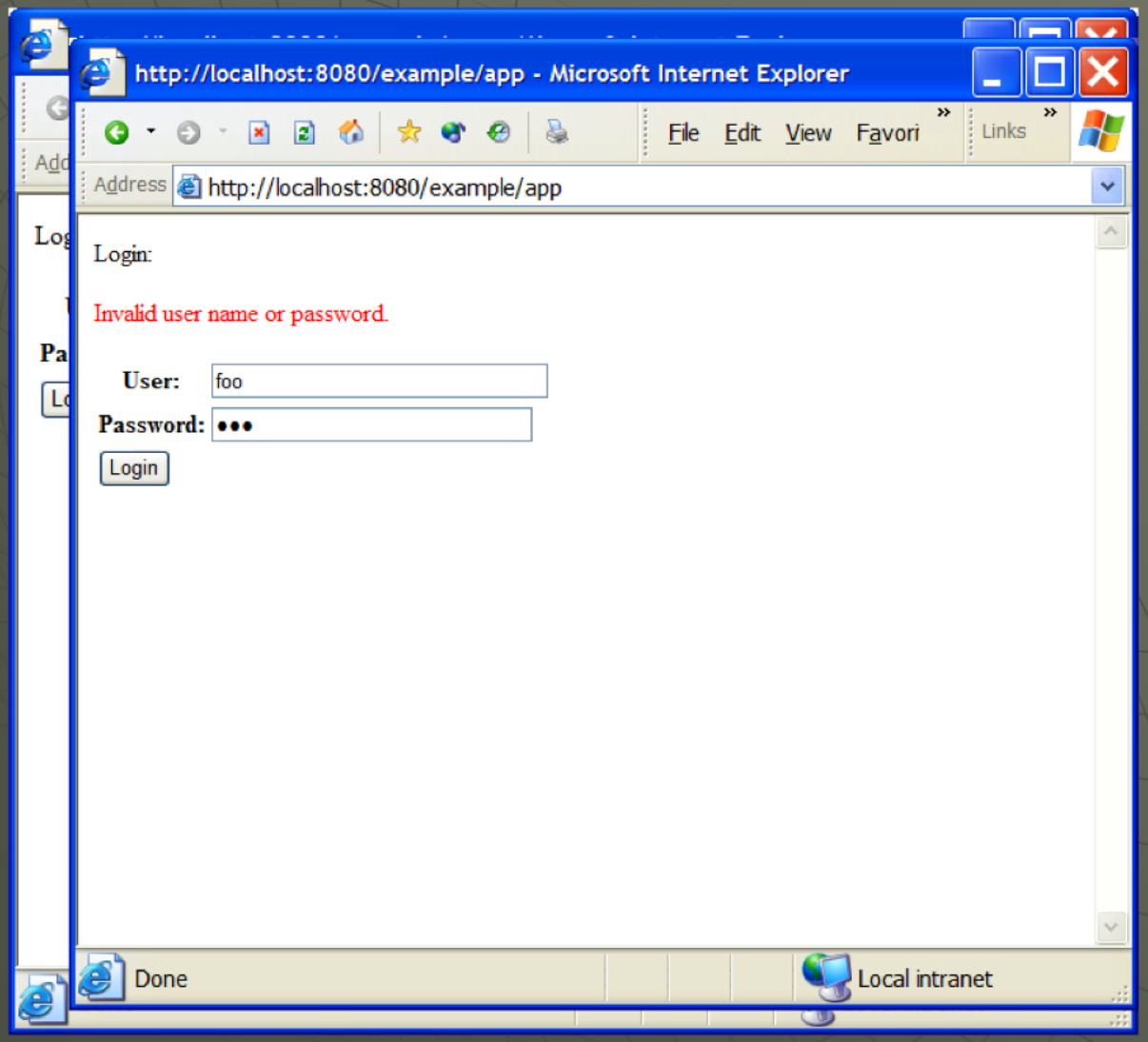
```
package example;

import org.apache.tapestry.IRequestCycle;
import org.apache.tapestry.html.BasePage;

public abstract class Home extends BasePage
{
  public void login(IRequestCycle cycle)
  {
    if (validLogin( getUserName(), getPassword()))
      cycle.activate("Main");
    else
      setMessage("Invalid user name or password.");
  }

  private boolean validLogin(String userName, String password)
  {
        // Talk to some backend system ...
  }
}
```

On failure, set error message

Current page stays active, renders response

message, userName and password are all filled in

Running The App



web.xml

```
<servlet>
  <servlet-name>example</servlet-name>
  <servlet-class>org.apache.tapestry.ApplicationServlet</servlet-class>
  <load-on-startup>1</load-on-startup>
</servlet>

<servlet-mapping>
  <servlet-name>example</servlet-name>
  <url-pattern>/app</url-pattern>
</servlet-mapping>
```

- Tapestry apps use a single servlet
- Servlet class provided by framework
- Usually mapped to "/app" within context
- http://localhost:8080/example/app

Simplicity

- Doing simple things should be simple
- Doing complex things should be simple
- Keep changes localized
- Example: Need a link on a page?
  - Add a link component to the HTML
  - Add a method to the Java class
- HTML templates simpler than JSPs
- Hide Servlet API – Make it irrelevant

Consistency

- Several ways to approach problems, one obvious best way
  - Good for teams
- Don't have to change approach as complexity of app changes
  - What works for simple apps, works for complex apps
  - Everything already in terms of fine-grained components

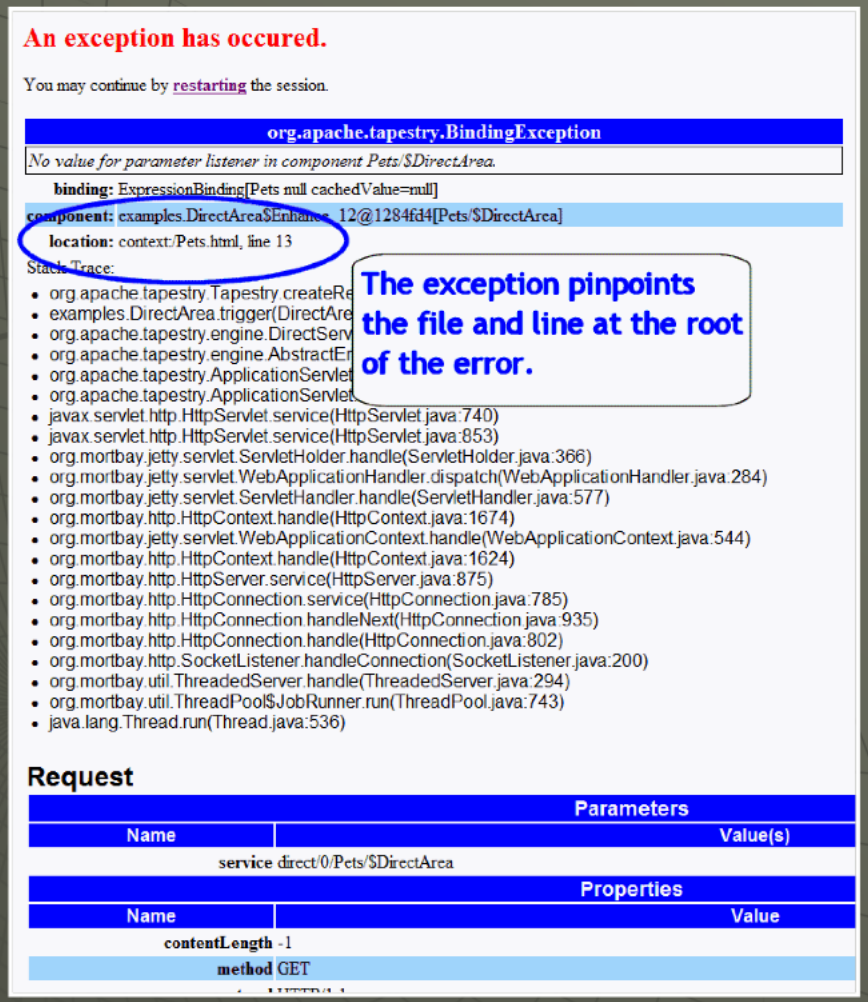
Efficiency

- Runtime efficiency through pooling, lazy object creation
- Development time efficiency by increasing Developer productivity
  - Less to code (simplicity)
  - No special cases (consistency)
  - Framework helps on errors (feedback)
  - More time to improve backend, the real performance hog

Feedback

- Errors in Tapestry NOT an afterthought
- Lots of checking throughout the framework
- Special effort on error messages
- *Line Precise Error Reporting*
  - Errors related back to template or specification file (right to the line)

Feedback



An exception has occured.

You may continue by restarting the session.

org.apache.tapestry.BindingException

No value for parameter listener in component Pets/$DirectArea.

binding: ExpressionBinding[Pets null cachedValue=null]

component: examples.DirectArea$Enhance...12@1284fd4[Pets/$DirectArea]

location: context:/Pets.html, line 13

Stack Trace:

- org.apache.tapestry.Tapestry.createRe...
- examples.DirectArea.trigger(DirectAre...
- org.apache.tapestry.engine.DirectServ...
- org.apache.tapestry.engine.AbstractEn...
- org.apache.tapestry.ApplicationServlet...
- org.apache.tapestry.ApplicationServlet...
- javax.servlet.http.HttpServlet.service(HttpServlet.java:740)
- javax.servlet.http.HttpServlet.service(HttpServlet.java:853)
- org.mortbay.jetty.servlet.ServletHolder.handle(ServletHolder.java:366)
- org.mortbay.jetty.servlet.WebApplicationHandler.dispatch(WebApplicationHandler.java:284)
- org.mortbay.jetty.servlet.ServletHandler.handle(ServletHandler.java:577)
- org.mortbay.http.HttpContext.handle(HttpContext.java:1674)
- org.mortbay.jetty.servlet.WebApplicationContext.handle(WebApplicationContext.java:544)
- org.mortbay.http.HttpContext.handle(HttpContext.java:1624)
- org.mortbay.http.HttpServer.service(HttpServer.java:875)
- org.mortbay.http.HttpConnection.service(HttpConnection.java:785)
- org.mortbay.http.HttpConnection.handleNext(HttpConnection.java:935)
- org.mortbay.http.HttpConnection.handle(HttpConnection.java:802)
- org.mortbay.http.SocketListener.handleConnection(SocketListener.java:200)
- org.mortbay.util.ThreadedServer.handle(ThreadedServer.java:294)
- org.mortbay.util.ThreadPool$JobRunner.run(ThreadPool.java:743)
- java.lang.Thread.run(Thread.java:536)

Request

Parameters	
Name	**Value(s)**
service	direct/0/Pets/$DirectArea

Properties	
Name	**Value**
contentLength	-1
method	GET

The exception pinpoints the file and line at the root of the error.

Feedback

- Uncaught exceptions get big report:
  - Entire stack of exceptions
  - Properties of each exception
  - Stack trace at *deepest* level
- Dumps out all the Servlet API objects
  - Servlet
  - HttpServletRequest
  - HttpSession
  - ServletContext
- Dumps out all JVM System Properties
- Result: tons of detail w/o using debugger!

Summary

- HTML still looks like HTML
- Properties read and updated by components
- Listener methods easy way to add functionality
- No worrying about URLs, query parameters
- Simplicity! Consistency!

Summary

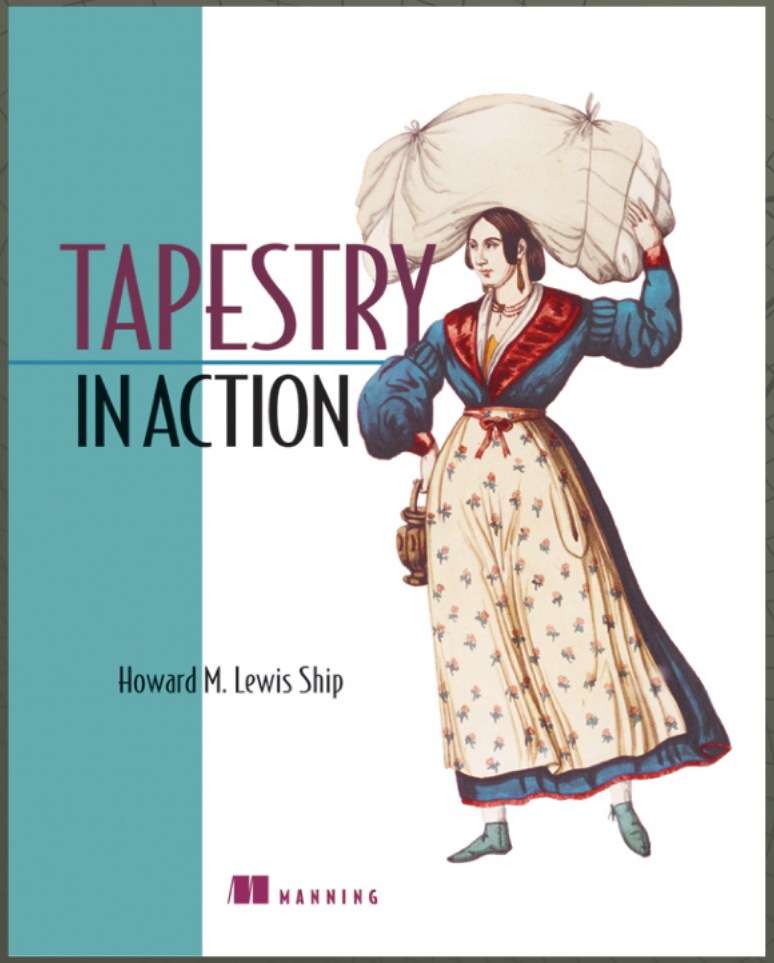
- Complexity very scalable
  - No "master descriptor" for team members to fight over
  - Naming conflicts between developers just don't happen
- HTML Dude vs. Java Guy
  - "Invisible markup" less prone to corruption
  - HTML can be previewed without running full application

Wrap Up

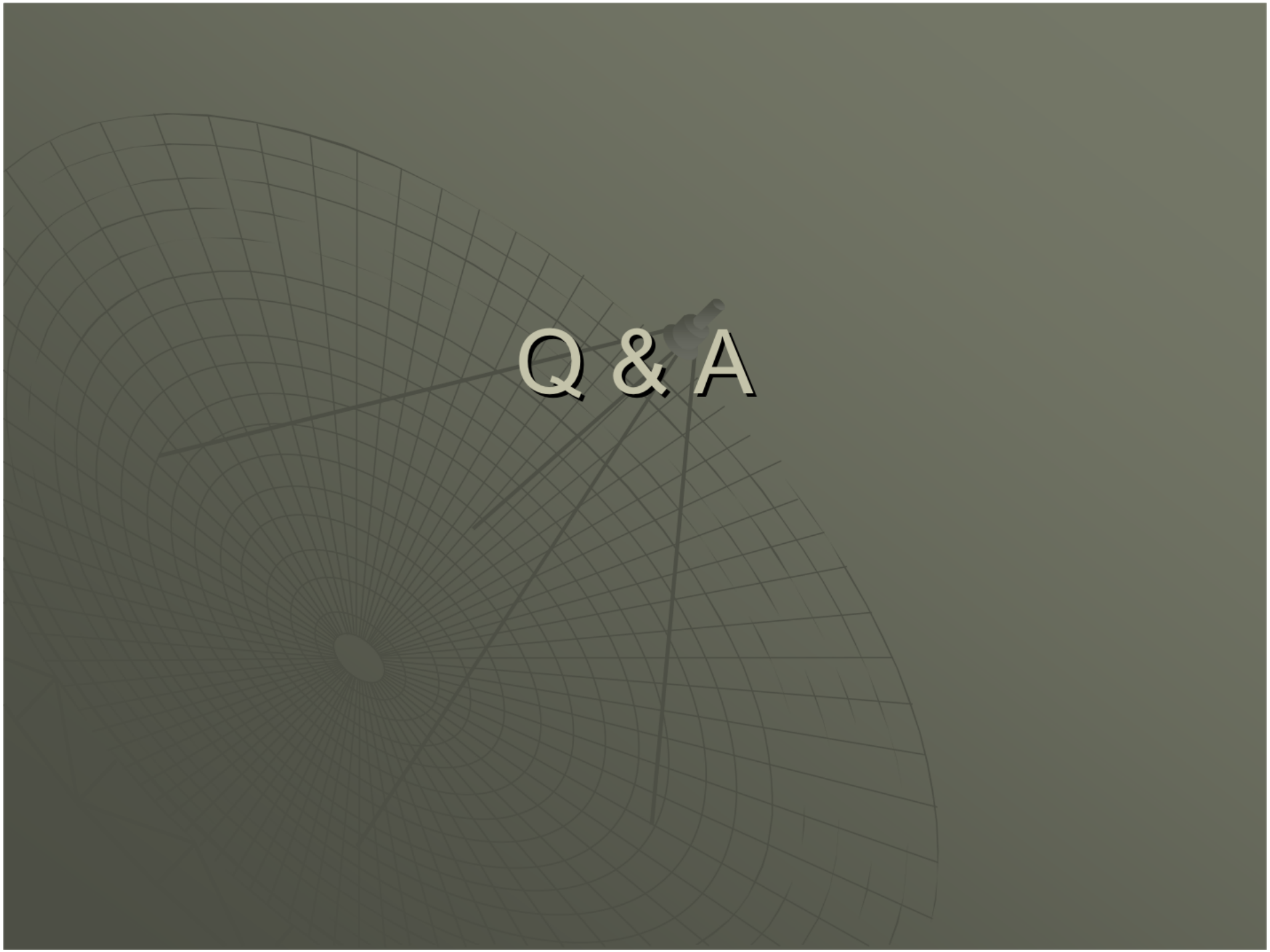
Home Page

- http://jakarta.apache.org/tapestry
- Documentation
- Downloads
- User and Developer Mailing Lists (w/ archives)

"Tapestry in Action"



- Published by Manning
- Available in print January 2004
- Covers Tapestry 3.0
- MEAP – Early Access PDFs
- ISBN 1932394117


Q & A

This presenation has ended.
Click anywhere to return.

_